Lab 02 — Questions

Answer without going back to the theory. Justify every answer: a claim without a mechanism counts for nothing.

Question 1 [Understanding]

For each of the names below, write out the **complete, explicit** name the engine builds from it. Explain how you can tell whether the first part is a registry or a namespace, and describe how Podman handles each short name:

```
nginx
bitnami/nginx
registry.mycompany.be:5000/base/nginx:1.25
```

Question 2 [Analysis]

Two servers, A and B, pulled `myapp/api:2.3` on the same day. Three weeks later, the team redeploys on B only, using the same command: `podman pull myapp/api:2.3`. A bug shows up on B but not on A. The version is "the same" — so how can that happen? Which command proves what happened, and which practice would have prevented the problem?

Question 3 [Diagnosis]

A developer notices that `podman images` lists 40 images adding up to 62 GB in the `SIZE` column, yet their WSL disk holds only 100 GB and has never run short of space. Are 62 GB of images really there? Explain, give the command that shows the real figure, and say where those files physically live on the workstation.

Question 4 [Analysis]

A Dockerfile contains:

```
COPY credentials.json /tmp/credentials.json
RUN ./configure.sh && rm /tmp/credentials.json
```

The author claims the secret is not in the final image, since they deleted it. Are they right? Explain precisely what the image contains, and why the `rm` does not help at all.

Question 5 [Understanding]

You push a second version of your Spring Boot image to the registry. It is 310 MB; the previous one was 308 MB. Yet the `push` transfers only 61 MB. Explain the mechanism, then say what you would need to change in your Dockerfile if the push transferred the full 310 MB every time.

Question 6 [Diagnosis]

```
$ podman images
REPOSITORY          TAG         IMAGE ID       SIZE
localhost/api       2.0         f3a1b9c02d11   310 MB
localhost/api       1.9         f3a1b9c02d11   310 MB
<none>              <none>      8b2c74e91a03   295 MB
```

Comment on this output. How many distinct images are actually there? Where does the `<none>` line come from? What exactly happens when you type `podman rmi api:1.9`? And why the `localhost/` prefix?

Question 7 [Analysis]

Your colleague builds the back-end image on their MacBook M3 and pushes it to the registry. Deployment on the staging server fails with `exec /usr/bin/java: exec format error`. Diagnose the failure and give two fixes: one to unblock the deployment right away, and one that stops the problem from ever recurring.

Question 8 [Understanding]

`podman save` and `podman export` both produce a `.tar` archive. In which situation is each one the right choice? What exactly do you lose if you use `export` to carry a Spring Boot image to an isolated site? And what does `--format oci-archive` change about a `save`?

Question 9 [Analysis]

After a `podman pull` of a 400 MB image, you run the same command again. It finishes in under a second. What did the engine actually verify — and why did that cost almost no network traffic?

Question 10 [Diagnosis]

Your company's CI fails intermittently on `podman pull node:22-alpine` with the message `toomanyrequests: You have reached your pull rate limit`. Nothing in the pipeline has changed. Explain the cause, explain why it strikes "intermittently", and give the two standard remedies companies apply.

Question 11 [Diagnosis]

You start a local registry (`podman run -d -p 5000:5000 registry:2`), then:

```
$ podman push localhost:5000/base/demo:1.0
Error: … pinging container registry localhost:5000: Get "https://localhost:5000/v2/":
http: server gave HTTP response to HTTPS client
```

Your colleague, who runs Docker, has never seen that message with the same registry. Explain how the two engines differ in attitude, give two ways to make the `push` succeed — and say why one of the two must never end up in a version-controlled configuration file.

Question 12 [Diagnosis]

A `podman rmi my-api:1.0` returns:

```
Error: image used by 4c2e9a1b7d33…: image is in use by a container: consider listing
external containers and force-removing image
```

Explain the situation and why the engine refuses. Give the **clean** way to resolve it — then say what `podman rmi -f` does and why it is a bad idea here.

Question 13 [Analysis]

`podman history` on an image shows several layers of size `0B` and one layer of `180MB`. What are the `0B` layers, and why do they exist at all? How does reading this output help you shrink an image in practice?

Question 14 [Understanding]

Your team is weighing three tagging strategies for the back-end image: (a) `api:latest`, overwritten at every build; (b) `api:1.4.2`, following the application version; (c) `api:1.4.2-b318-a9f3c21`, including the build number and the Git *commit*. Compare all three in terms of **production rollback** and **incident diagnosis**.